Report Progetto

Daniele Di Renzo **Valerio Da Corte** **Andrei Valentin Chircu**

1 Obiettivo del Progetto

Il progetto realizza una pipeline containerizzata per l'acquisizione e l'estrazione di testo pulito da sorgenti web eterogenee tramite parser dedicati per dominio. Il sistema espone i dati tramite un'API REST (FastAPI) e una Web UI interattiva, valutando automaticamente la qualità del testo estratto rispetto a un Gold Standard costruito manualmente.

2 Organizzazione del Codice

Il codice segue un'architettura multi-container orchestrata con Docker Compose, con separazione netta di responsabilità tra i moduli.

Server REST (`backend/src/server.py`). Implementa tutti gli endpoint FastAPI: `GET/POST /parse`, `GET /domains`, `GET /gold_standard`, `GET /full_gold_standard`, `POST /evaluate` e `GET /full_gs_eval`. I due metodi `/parse` gestiscono rispettivamente il parsing online (via `GET`) e il parsing da HTML locale già scaricato (via `POST`).

Valutazione (`backend/src/evaluation.py`). La classe `Evaluator` implementa le metriche di valutazione a livello token (`token_level_eval`) e le metriche aggiuntive (`x_eval`). La funzione `evaluate_gold_standard` orchestra la valutazione su tutto il Gold Standard, selezionando il parser corretto per dominio.

Parser (`backend/src/parsers/`). `BaseParser` definisce la logica comune: crawling con Crawl4AI, metodo `remove_markdown` per convertire Markdown in testo piano, e il metodo `run_from_html` per il parsing da HTML locale. Le sottoclassi `WikipediaParser`, `IlPostParser`, `StatMuseParser` ed `EurLexParser` sovrascrivono `create_crawler_config`, `extract_title`, `parse_markdown` e `format_result` con selettori CSS e logica di post-elaborazione specifici per ciascun dominio.

Frontend (`frontend/src/frontend.py`). Server FastAPI con rendering Jinja2 che offre una Web UI per inserire URL, confrontare HTML grezzo con il testo parsato e visualizzare le metriche di valutazione quando è disponibile un Gold Standard per quell'URL.

Gold Standard (`gs_data/`). Per ogni dominio è presente un file `gs.json` con i metadati delle 10 entry (URL, dominio, titolo, percorsi ai file) e una cartella `texts/` con i testi lunghi. Il contenuto testuale è referenziato tramite percorso a file esterno anziché inline nel JSON, per mantenere il file leggero e facilmente versionabile.

Orchestrazione (`docker-compose.yaml`). Avvia backend (porta `8003`) e frontend (porta `8004`) con un unico comando `docker compose up --build`. Il frontend è configurato con `depends_on: condition: service_healthy`, garantendo che si avvii solo dopo che il backend ha superato il proprio healthcheck HTTP.

3 Valutazione dei Parser

3.1 Metodologia

Per ogni dominio sono state selezionate 10 URL rappresentative; il testo informativo (titolo e corpo, esclusi menu, footer e sidebar) è stato copiato manualmente per costruire il Gold Standard.

La metrica principale `token_level_eval` confronta i token (parole minuscole, punteggiatura rimossa) dei due testi usando `Counter`, che tiene conto delle ripetizioni anziché operare su insiemi puri. Questa scelta è deliberata: un parser che duplica interi paragrafi verrebbe penalizzato corret-

tamente sulla Precision, mentre con un approccio a insiemi tale difetto passerebbe inosservato.

3.2 Risultati

Dominio	Prec.	Rec.	F1
en.wikipedia.org	0.993	0.986	0.989
www.ilpost.it	0.999	1	1
www.statmuse.com	0.999	1	0.999
eur-lex.europa.eu	0.969	1	0.984

Table 1: Medie `token_level_eval` per dominio (post-ottimizzazione).

I risultati mostrano un'elevata qualità di estrazione su tutti i domini, con F1 sempre superiore a 0.98. Il valore di Precision leggermente inferiore per EUR-Lex (0.969) riflette la presenza di elementi strutturali non sempre eliminabili con i soli selettori CSS.

3.3 Scelte Implementative

Per garantire riproducibilità e velocità, la valutazione del Gold Standard avviene, se disponibili, su file HTML salvati localmente, altrimenti, in caso di fallback, l'HTML viene acquisito direttamente facendo crawling sull'URL durante la valutazione. La differenza chiave tra i domini riguarda come questi file vengono ottenuti.

Caso generale — sorgente HTTP statico. Per la maggior parte dei domini (Wikipedia, Il Post, EUR-Lex) il testo informativo è interamente presente nella risposta HTTP iniziale grazie al Server-Side Rendering: è stato sufficiente salvare il sorgente HTML così com'è, senza eseguire JavaScript. Nel caso specifico di `en.wikipedia.org`, l'uso del browser headless risultava anzi controproducente: l'esecuzione di JavaScript iniettava nel DOM elementi estranei (popup di anteprima, widget di accessibilità, classi `client-js`) assenti nel Gold Standard, causando un calo di Precision. Passando al sorgente statico, l'F1 è salito da 0.9859 a 0.9887, i tempi di acquisizione si sono ridotti drasticamente e i selettori CSS sono diventati più stabili.

Eccezione — StatMuse e il rendering client-side. `statmuse.com` rappresenta il caso opposto: i dati statistici vengono iniettati nel DOM via API *dopo* il rendering della pagina e sono quindi assenti nel sorgente HTTP. Per questo dominio l'HTML locale è stato necessariamente ottenuto tramite crawling con browser headless, l'unico modo per eseguire il JavaScript e ottenere il DOM completo. Sull'HTML così acquisito è stata poi applicata una fase di post-elaborazione con BeautifulSoup per estrarre sequenzialmente titoli e tabelle, con deduplicazione per eliminare le tabelle duplicate generate dai breakpoint CSS responsive.

3.4 Metriche Aggiuntive (extra)

Accanto a Precision/Recall/F1, sono implementate cinque metriche aggiuntive raggruppate in `x_eval`. Per gestire il loro diverso costo computazionale, sono disponibili in due varianti: `x_eval_fast` ($O(N+M)$), usata nella valutazione completa del Gold Standard dove si valutano 10 URL per dominio, e `x_eval_complete` (fino a $O(N \times M)$), disponibile nella Web UI per la valutazione di singole URL dove la latenza è accettabile.

Cosine Similarity [*fast*]. Rappresenta ciascun testo come vettore di frequenze (TF) e ne misura l'angolo nel spazio vettoriale. A differenza dell'F1, non è influenzata dalla lunghezza del testo: due articoli sullo stesso argomento con vocabolario simile otterranno un punteggio alto anche se uno è molto più lungo dell'altro, rendendola una buona misura di somiglianza tematica.

Boilerplate Ratio & Fidelity [*fast*]. Il *Boilerplate Ratio* è il rapporto tra caratteri alfanumerici e caratteri totali nel testo estratto: un valore basso segnala simboli, spazi eccessivi o markup residuo. La *Boilerplate Fidelity* è il complemento a 1 della differenza assoluta tra il Boilerplate Ratio del testo estratto e quello del Gold Standard: misura quanto il parser preserva la densità informativa della sorgente, penalizzando sia l'estrazione di rumore sia la perdita eccessiva di contenuto.

Levenshtein normalizzato [*complete*]. Calcola il numero minimo di operazioni (inserimenti, cancellazioni e sostituzioni di token) necessarie per trasformare la sequenza estratta in quella di riferimento, normalizzato in $[0, 1]$. Rispetto all'F1, penalizza esplicitamente le sostituzioni: due testi con lo stesso numero di token ma vocabolario diverso ricevono un punteggio basso, mentre l'F1 potrebbe comunque risultare alto se i token in comune sono molti. Il costo $O(N \times M)$ lo rende impraticabile sulla valutazione dell'intero Gold Standard, motivo per cui è incluso solo in `x_eval_complete`.

ROUGE-L [*complete*]. Basata sulla Longest Common Subsequence (LCS) tra i token, valuta se l'ordine delle parole è preservato. È utile per

rilevare parser che estraggono il contenuto corretto ma in ordine alterato, ad esempio per sorgenti con layout a colonne o contenuti riorganizzati dal renderer JavaScript. Analogamente a Levenshtein, il costo $O(N \times M)$ della LCS lo confina in `x_eval_complete`.

Nel complesso, le cinque metriche sono state scelte per coprire aspetti complementari della qualità: contenuto (*token_level_eval*, *Cosine Similarity*), struttura e ordine (*ROUGE-L*, *Levenshtein*) e pulizia del testo estratto (*Boilerplate Ratio & Fidelity*).